INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA
SALT LAKE
(Affiliated to UEM Kolkata and Approved by AICTE)

Advanced Programming Lab
[PCCCS495]

Hospital Management System

Name: -ARIJIT DEB

Semester: -IV

Class Roll: - 75

Enrolment Number: - 12024052004072

Date: - 29/03/2026

Instructor name: - Susovan Jana, Soumadip Biswas, Swagatam Basu

Department of Information Technology

2026

2. Problem Statement

In real-world reservation systems such as railway or airline booking platforms, multiple users attempt to book seats simultaneously. This creates challenges such as **race conditions**, **double booking**, and **data inconsistency**.

The objective of this project is to design and implement a **multithreaded seat reservation system** using Java that:

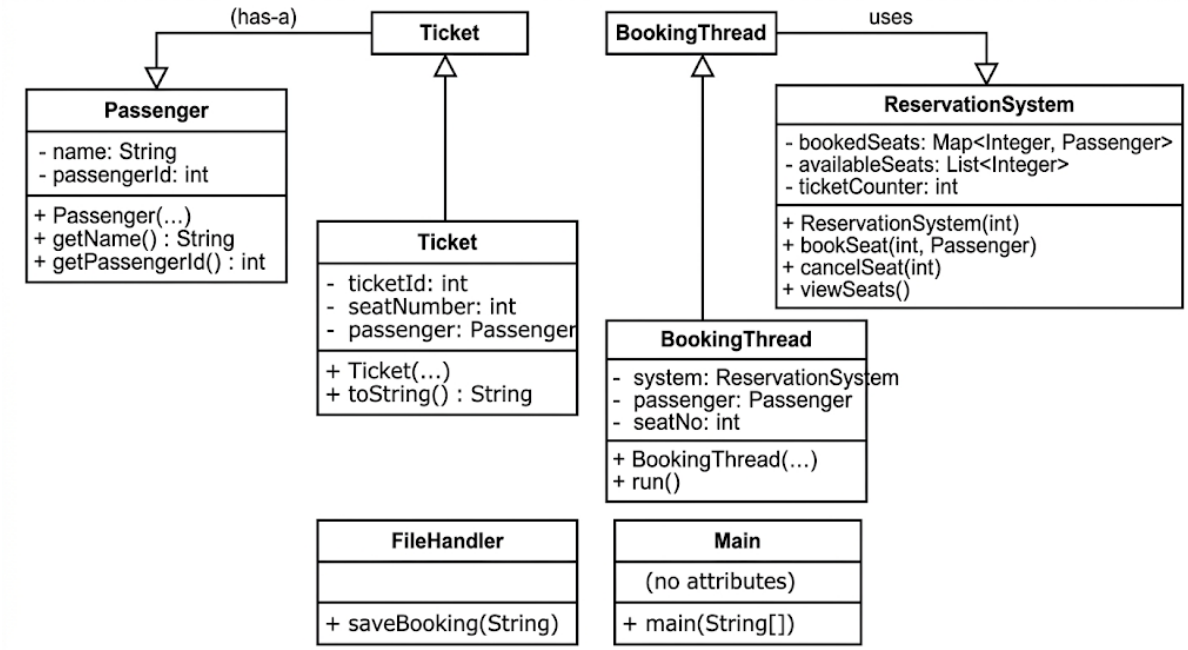
- Allows booking and cancellation of seats
- Prevents double booking using synchronization
- Handles concurrent access safely
- Stores booking data persistently in a file

The system simulates a real-world booking environment using **threads**, **object-oriented design principles**, and **file handling mechanisms**.

---

3. System Design

3.1 UML Class Diagram (Mandatory)



Classes:

- Passenger
- Ticket
- ReservationSystem
- FileHandler
- BookingThread
- Main

Relationships:

- Ticket → Passenger (HAS-A)
- ReservationSystem → Passenger (USES)
- BookingThread → ReservationSystem (INTERACTS)

3.2 Description of Major Classes

Passenger Class

- Stores passenger details like name and ID
- Acts as a simple data model

Ticket Class

- Represents a booked ticket
- Contains ticket ID, seat number, and passenger details

ReservationSystem Class

- Core logic of the system
- Manages booking, cancellation, and seat tracking
- Uses:
    - HashMap for booked seats
    - ArrayList for available seats
- Uses synchronization for thread safety

FileHandler Class

- Handles file operations
- Saves booking details into bookings.txt

BookingThread Class

- Implements multithreading
- Each booking runs as a separate thread

Main Class

- Provides menu-driven interface
- Takes user input and triggers operations

3.3 Package Structure

```
src/
│── Passenger.java
│── Ticket.java
│── ReservationSystem.java
│── FileHandler.java
│── BookingThread.java
│── Main.java
```

3.4 Interaction Flow

1. User selects booking option
2. Passenger object is created
3. Thread is initiated
4. ReservationSystem checks availability
5. Seat is booked if available
6. Ticket is generated
7. Booking is saved to file

---

4. OOP Concepts Demonstrated

4.1 Encapsulation

Where used: Passenger and Ticket classes
Why used: To protect data and provide controlled access
Code Snippet:

```
public class Passenger {
    private String name;
    private int passengerId;

    public String getName() {
        return name;
    }
}
```

Explanation:
Data members are private and accessed through getter methods, ensuring controlled data access.

4.2 Abstraction

Where used: ReservationSystem methods
Why used: Hides internal booking logic from users
Code Snippet:

```
public synchronized void bookSeat(int seatNo, Passenger p) {
    if (!availableSeats.contains(seatNo)) return;
    availableSeats.remove(Integer.valueOf(seatNo));
}
```

Explanation:
User interacts with simple methods without knowing internal seat management logic.

4.3 Inheritance

Where used: BookingThread extends Thread
Code Snippet:

```
public class BookingThread extends Thread {
    public void run() {
        system.bookSeat(seatNo, passenger);
    }
}
```

Explanation:
Inheritance allows thread behavior to be customized for booking operations.

4.4 Polymorphism
Where used: Method overriding (run() method)
Explanation:
Different thread objects execute the same method differently based on input.

---

5. Implementation Highlights

Snippet 1: Seat Booking Logic

```
public synchronized void bookSeat(int seatNo, Passenger p) {
    if (!availableSeats.contains(seatNo)) {
        System.out.println("Seat already booked!");
        return;
    }
    availableSeats.remove(Integer.valueOf(seatNo));
    bookedSeats.put(seatNo, p);
}
```

Justification:
Ensures thread-safe booking and prevents duplicate reservations.

Snippet 2: Seat Cancellation

```
public synchronized void cancelSeat(int seatNo) {
    if (!bookedSeats.containsKey(seatNo)) return;
    bookedSeats.remove(seatNo);
    availableSeats.add(seatNo);
}
```

Justification:
Allows reuse of cancelled seats efficiently.

Snippet 3: File Writing

```
BufferedWriter bw = new BufferedWriter(
    new FileWriter("bookings.txt", true));
bw.write(data);
bw.newLine();
```

Justification:
Ensures persistent storage of booking records.

Snippet 4: Multithreading

```
Thread t = new Thread(() -> {
    system.bookSeat(seat, p);
});
t.start();
```

Justification:

Simulates multiple users booking seats concurrently.

---

6. Testing & Error Handling

Test Cases Considered

- Booking an available seat
- Booking an already booked seat
- Cancelling a booked seat
- Cancelling an unbooked seat

Edge Cases Handled

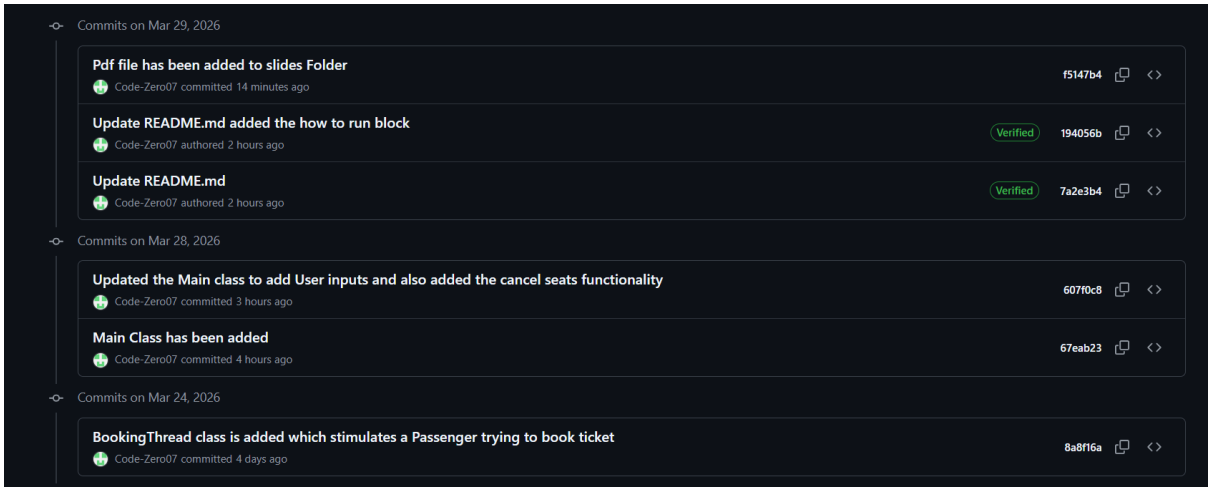
- Invalid seat number
- Duplicate booking attempts
- Empty seat list

Failure Scenarios

- File write failure → handled using try-catch
- Concurrent access → handled using synchronized
- Invalid input → basic validation applied

---

7. Git Workflow



Development Progression Explanation

The project was developed in a **step-by-step modular approach**, where each component was implemented and tested incrementally. This ensured better code organization, easier debugging, and proper version control.

Explanation of Development Progression

- **Phase 2: Core Data Models (Mar 22, 2026)**
  Passenger class added
  Ticket class added
  - *Purpose:*
    Defined fundamental entities of the system
    Established data structures for passengers and tickets
  - *Impact:*
    Enabled representation of users and booking information

- **Phase 3: System Core Development (Mar 23, 2026)**
  ReservationSystem class added (core logic)
  FileHandler class added (file storage)
  Minor duplicate commit fix for FileHandler
  - *Purpose:*
    Implemented seat booking and cancellation logic
    Introduced file handling to persist booking data
  - *Impact:*
    System became functionally capable of managing reservations

---

- **Phase 4: Concurrency Implementation (Mar 24, 2026)**
  BookingThread class added
  - *Purpose:*
    Simulate multiple users booking seats simultaneously
    Introduce multithreading into the system
  - *Impact:*
    Improved realism of the system
    Enabled testing of synchronization and race condition handling

---

- **Phase 5: User Interaction Layer (Mar 28, 2026)**
  Main class added

Added:
    - User input functionality
    - Menu-driven interface
    - Seat cancellation feature
  - *Purpose:*
    Provide an interactive interface for users
    Allow dynamic testing of booking and cancellation
  - *Impact:*
    System became fully usable from the console

- **Phase 6: Final Enhancements (Mar 28, 2026)**
    Updated Main class with improved input handling
    Refined cancellation functionality
  - *Purpose:*
    Improve usability and correctness
    Handle edge cases more effectively

- **Phase 7: Documentation & Finalization (Mar 28, 2026)**
    README.md updated
  - *Purpose:*
    Document project details
    Prepare for submission and evaluation

---

8. Conclusion & Future Scope

Conclusion
This project successfully demonstrates a **thread-safe reservation system** using Java. It highlights the importance of synchronization, object-oriented design, and concurrency in real-world applications.

Future Scope
- GUI implementation using JavaFX or Swing
- Database integration (MySQL)
- Online web-based booking system
- Real-time seat tracking
- Payment gateway integration

9.Appendix

Review Mail

approval email with APPROVED_MINOR

APPROVAL_IMAGE

Additional Submissions (if any)

N.A